

Titan Hardware Manager v3

15-Hour Continuous Stress Test

Results, New Findings, and Fixes

Report date	2026-10-09
Test window	2026-10-08 23:43:11 to 2026-10-09 14:43:38 IST (15 h 0 m 27 s)
Component under test	Titan Hardware Manager v3 (THM v3) — classify / detect / tick / policy / decision pipeline
Harness binary	titan-hwm-v3/build/thm_stress_test (tester tree)
Driver script	scripts/run_15h.sh
Run commit / analysis	built at 8f1c052; analysed at fb28b06
Raw data	stress_15h.csv (5,364 samples), run.log, ANALYSIS.md, RUN_STATUS
Data location	/home/msfvenom/thm-stress-15h/ on nvme0n1p2 (SSD)
Overall verdict	ALL 5 ASSERTIONS PASSED — the first run to reach the 100,000-tick latency-buffer cap, where the memory curve flattens

Executive summary

The 15-hour continuous stress test ran to completion as a single process for 268,201 ticks and **passed all five assertions**. It is the first campaign in the series long enough to cross the 100,000-tick point at which the harness's own latency ring buffer saturates. At that point the memory curve flattens: RSS rose to a plateau of roughly 2.5 MB over the first 5.6 hours and then held flat — and even drifted gently down — for the remaining 9.4 hours. This directly refutes the unbounded-leak reading that motivated the original ISSUE-01 finding. The pipeline executed 53.4 million decisions, performed 16,174 reclaimings with zero aborts, and never exceeded a 32 ms tick against a 1,500 ms ceiling.

Three results matter:

- **The memory growth is a harness artefact, now confirmed — not assumed.** The growth rate is constant at 16 bytes per tick until the buffer cap is reached, then goes to zero. A genuine 430 KB/h leak would have reached 6,450 KB of growth by 15 hours; the run observed 1,608 KB above its first CSV sample and plateaued.
- **PID handling is leak-free at the scale tested.** System-wide PID count added 67 net over 15 hours, consistent with the 6-hour run and isolating the 10-hour stage-1 figure as the anomaly.
- **The 15-hour run measured the harness, not the shipped daemon.** At the time it ran the harness was decision-only and never called `EnforcementPlane::apply()`, so this run covers no kernel enforcement. The harness can now observe enforcement — an F1 probe was added afterward and verified as root.

This report also records the **seven new findings** surfaced while preparing, running, and interpreting the soak. Five are now resolved: two shipping defects in the systemd unit (a `--dry-run` directive that leaked into the wrong copy, and an invalid user-session target on a root system service) are fixed, and three harness defects (F1, F4, F6) were implemented, verified, and pushed to the tester tree after this report's first draft. F5 and F7 remain open.

Test configuration

Executed by `scripts/run_15h.sh` as one invocation of the tester binary. No staging and no restarts — the process count never resets, so the 100,000-tick cap is reachable.

Parameter	Value
<code>--duration</code>	54,000 s (15 h)
<code>--tick-ms</code>	200 ms per tick
<code>--force-pressure</code>	CRITICAL
<code>--aging-phase</code>	enabled
<code>--memory-floor-mb</code>	2048

--assert	1500 ms
--pid-budget	45,000
--rss-budget-pct / --rss-budget-kb	200 % / 4,096 KB
Privilege	uid 1000; archtitan.slice cgroup absent → cgroup enforcement is a decision-only no-op

The run executed with a 2 GB memory floor, which forces real reclaim and lets PSI register (the ISSUE-07 fix). This has a consequence for interpretation: the mem_used_pct and psi_some columns are **not** comparable to the pre-floor campaigns. It is treated as a limitation, not a regression.

Assertion results

#	Assertion	Result
1	RSS growth 2,516 KB (53 %) within 200 % / 4,096 KB budget	PASS
2	System PID growth −66 within budget 45,000	PASS
3	Zero reclaim aborts across 16,174 reclaims	PASS
4	Tick p95 32 ms ≤ 1,500 ms ceiling	PASS
5	Decision throughput OK (53,432,500 decisions)	PASS

Assertion 2 records −66 because its baseline is a single sample taken at process start (before the workload pool had settled); the CSV trend, measured from the first logged sample, is +67. Both are far inside budget. The discrepancy is a measurement artefact and is logged as finding F6.

Headline result — the RSS plateau

The saturation model predicts roughly 2.4 MB (2,400 KB) of buffer growth that stops at tick 100,000. An unbounded leak predicts 6,450 KB of growth by 15 hours. Observed, from process start: **2,516 KB** — a ratio of **1.07** to the model — with the rate dropping to zero exactly at the cap.

Growth per 10,000 ticks

Growth is constant at 16 B/tick (about 160 KB per bucket) and stops at the five-digit boundary. PID growth is included to show it is flat throughout.

Ticks	RSS Δ KB	PID Δ	Ticks	RSS Δ KB	PID Δ
0–9,999	0	0	140,000–149,999	1,776	69
10,000–19,999	400	72	150,000–159,999	1,776	71
20,000–29,999	560	69	160,000–169,999	1,776	72
30,000–39,999	712	69	170,000–179,999	1,696	72
40,000–49,999	872	69	180,000–189,999	1,656	66
50,000–59,999	1,024	71	190,000–199,999	1,656	66
60,000–69,999	1,184	66	200,000–209,999	1,624	68
70,000–79,999	1,336	70	210,000–219,999	1,608	68
80,000–89,999	1,464	71	220,000–229,999	1,608	67
90,000–99,999	1,616	67	230,000–239,999	1,608	67
100,000–109,999	1,776	70	240,000–249,999	1,608	69
110,000–119,999	1,776	70	250,000–259,999	1,608	67
120,000–129,999	1,776	69	260,000–269,999	1,608	67
130,000–139,999	1,776	70			

The right-hand column is the decisive evidence: from tick 210,000 onward the RSS figure does not move. The leak hypothesis is refuted by the run's own data, once and for all, rather than by extrapolation.

PID, reclaim, latency, throughput

Metric	Value
Samples	5,364
Elapsed	53,993 s (15.00 h)
Ticks	268,201
Decisions	53,432,500
Reclaims / aborts	16,174 / 0
Tick p95	32 ms (ceiling 1,500 ms)
System PID — first CSV sample	427
System PID — final	494 (peak 629)
System PID — net, CSV	+67
System PID — net, assertion baseline	−66

total_pids counts every numeric entry in /proc, i.e. the whole desktop, not the THM tree. On a non-isolated machine that figure is environment-sensitive; what it shows reliably is **no accumulation**. Finding F6 proposes counting only the daemon's own cgroup.

Pressure and swap under the memory floor

Metric	Value
Peak memory used	91.9 %
PSI > 0	96 of 5,364 samples (1.8 %), peak 2.50
Last positive PSI sample	t = 48,977 s (13.6 h in)
Swap	1.7 GB held; so ≈ 0 throughout

The floor forces the surplus into swap, but the swap is **held, not thrashed**: the page-out rate stays near zero, which is why PSI remains low even under CRITICAL pressure. This is the expected signature of a hard floor, and it is the reason the PSI figures cannot be read as a stall measurement — see finding F5.

New findings and fixes

Seven findings were surfaced during this run. Two are shipping defects in the systemd unit and were fixed immediately; three more (F1, F4, F6) were implemented and verified in the tester tree after the run. Two remain open.

ID	Finding	Type	State
F1	Harness never applies enforcement (decision-only)	Test defect	Fixed
F2	- - dry-run leaked into the product service unit	Ship defect	Fixed
F3	Root system service targeted a user-session target	Ship defect	Fixed

F4	<code>setup_slices()</code> silently no-ops if the slice is inactive	Observability	Fixed
F5	<code>--aging-phase</code> suppresses PSI and hides stall pressure	Test design	Open
F6	PID assertion baseline is one noisy startup sample	Measurement	Fixed
F7	<code>reap_pid</code> blocks in <code>waitpid</code> on a SIGKILLED child	Latent risk	Open

F1 — the harness never applies enforcement

Finding. `tests/stress_test.cpp` calls `cgroup.setup_slices()` but never `move_to_slice()`, `set_freeze()`, `set_cpu_weight()`, `set_memory_high()`, or `EnforcementPlane::apply()`. Every “decision” in every campaign was recorded and then discarded. This is why ISSUE-06 has never had empirical coverage: the test is physically incapable of producing it.

Impact. High. It invalidates any claim that enforcement “works”; only the policy/classification layer has been exercised.

Solution. Fixed and verified. The harness gained `--enforce-probe`, which forks a sacrificial child, calls `EnforcementPlane::apply()` with a `FREEZE` decision against it, and asserts the kernel actually acted: the process state must be `T` and the frozen slice’s `cgroup.freeze` must read 1, after which the child is thawed and reaped. Run as root under `Slice=archtitan.slice`, the probe passes — child state `S→T`, `cgroup.freeze=1`, `apply()=true`. This is the first end-to-end observation of kernel-level enforcement in the series. In a decision-only environment the probe SKIPS rather than fails, so it never blocks ordinary soak runs.

F2 — --dry-run leaked into the product service unit

Finding. Two copies of `titan-hwm.service` exist. The source-of-truth copy in `titan-hwm-v3/` carried `ExecStart=/usr/local/bin/titan-hwm-daemon --dry-run`; the copy that actually ships (`airootfs/etc/systemd/system/`) did not. A dry-run daemon enforces nothing. A future install that regenerated the unit from the source copy would have shipped a permanently inert manager.

Impact. High if triggered — a silent, total enforcement outage.

Solution. Fixed. The product-tree unit was reconciled to the shipping copy: `--dry-run` removed. The two files are now byte-identical.

F3 — root system service targeted a user-session target

Finding. The same source unit used `graphical-session.target` in its `After=`, `PartOf=`, and `WantedBy=` directives. `graphical-session.target` is a **user** session target; it does not exist in the system manager, so these directives were inert and the ordering guarantees they imply did not hold.

Impact. Medium — the service would start on the wrong schedule and could race the session at boot.

Solution. Fixed together with F2: all three directives changed to `graphical.target` (the system target), matching the shipping copy.

F4 — silent no-op when the slice is inactive

Finding. If `archtitan.slice` is not active, `setup_slices()` returns without error and the whole run proceeds as a decision-only no-op with no warning. This masked the fact that none of the ten-hour campaign’s `cgroup` writes ever happened.

Impact. Medium. It turns a configuration mistake into a silently invalid test.

Solution. Fixed. `setup_slices()` now verifies real write access with a `cgroup.freeze` write-probe instead of trusting that the sub-slice directories exist — under an undelegated slice the directories exist but every control write is `EACCES`, which is exactly how the ten-hour run’s writes disappeared without a trace. The harness now prints a `DECISION-ONLY` banner and a slice status line when the slice is unavailable, and aborts with `exit 2` under the new `--require-slice` flag. The daemon’s “hierarchy setup failed” warning now fires correctly in this case too.

F5 — --aging-phase suppresses PSI

Finding. The aging phase replaces one spawn in three with an idle process, which removes exactly the churn that generates memory stalls. Under CRITICAL with a 2 GB floor, PSI still only registered in 1.8 % of samples (peak 2.50). Reclaim **coverage** and sustained **pressure** are in tension.

Impact. Medium. PSI numbers from an aging run cannot be used to argue the system survives real stall pressure.

Solution. For pressure campaigns, either drop --aging-phase, raise the floor, or add a dedicated non-aging stage that sustains PSI above zero. Keep the aging run for reclaim-path coverage and label its PSI as non-diagnostic.

F6 — PID assertion baseline is a single noisy sample

Finding. The PID assertion subtracts one /proc scan taken at process start (560 on this host) from the final scan (494), reporting -66. The CSV, measured from the first ten-second sample, reports +67. Neither is a clean signal, because total_pids counts the entire desktop.

Impact. Low. The budget is wide enough that the verdict is unaffected, but the printed number is misleading and cannot be trended.

Solution. Fixed. The guard now measures the harness's own process tree (tree_pids, walked over PPID links) instead of the whole machine's /proc total. The system-wide figure is still reported, but labeled informational. tree_pids is appended to the CSV, so existing columns and analyzers are unchanged.

F7 — reap_pid blocking wait (latent)

Finding. reap_pid blocks in waitpid until a SIGKILLED child actually exits; a child that has been swapped out or is in uninterruptible sleep can stall the tick loop. Not observed at 15 hours, but the condition is reachable under heavier swap.

Impact. Low now, potentially high under memory exhaustion.

Solution. Use a bounded wait (WNOHANG with a deadline, escalating to a logged warning if the child does not exit), so a stuck child cannot stall the pipeline.

What this run does not cover

Three limits are material and should travel with any citation of these numbers:

- **It measures the harness, not the daemon.** Every figure above describes thm_stress_test.titan-hwm-daemon has never been profiled under load.
- **This run observed no enforcement action.** At the time it ran the harness was decision-only (F1); the enforcement probe that now proves the kernel path was added afterward, so the 15-hour numbers themselves still cover no enforcement. Throttle and memory.high remain unexercised even by the probe, which only verifies FREEZE.
- **PSI and memory-usage figures are floor-specific.** The 2 GB floor makes them non-comparable to the pre-fix campaigns, and --aging-phase (F5) makes the PSI numbers non-diagnostic.

Recommended next actions

1. **Verify THROTTLE and memory.high, not just FREEZE.** The F1 probe proves the freeze path; cpu.weight and memory.high still have no observed effect.
2. **Run the daemon as root, non-dry-run, under Slice=archtitan.slice, and log one enforcement action on a real workload.** The harness probe covers the path; the shipped daemon itself is still untested under load.
3. **Profile the daemon's own RSS and latency under sustained load.** Every measurement to date is the harness.
4. **Keep the product and shipping systemd units identical, and add a check that rejects a --dry-run unit (F2/F3)** — done by hand here, worth automating.
5. **Add a non-aging pressure stage that sustains PSI above zero (F5).**
6. **Bound reap_pid's wait with a deadline (F7).**